

"Computer Science is a science of abstraction – creating the right model for a problem and devising the appropriate mechanizable techniques to solve it."

- A. Aho and J. Ullman

CSE 312

Operating Systems

Spring 2026

Inter-process Communication

© 2013-2026 Yakup Genç

1

Today

- Inter-process Communication

2

Inter-process Communication

- Frequent need for communication between processes (or threads)
- Three possible issues
 - How to pass information from one to the other
 - Easier for threads
 - Preventing hold ups
 - Proper sequencing under dependencies

3

A Write Operation in the Kernel

```
queued_len += user_len;  
if (queued_len == user_len)  
    startio();
```

If there's no I/O in progress start it up

4

But an Interrupt Happens in the Middle

```
queued_len += user_len;  
                                queued_len -= io_block_len;  
if (queued_len == user_len)  
startio();
```

The **if** statement will fail to start the I/O

Spring 2026

Operating Systems

5

5

What's the Problem?

- With interrupts, code execution is not linear
- Two different flows of execution accessed the same variable at about the same time
- We need a way to prevent that
- As always, multiprocessors makes this much worse

Spring 2026

Operating Systems

6

6

Race Condition

- Undesirable behavior that can occur from inappropriate reliance on ordering of operations
- When can Race Conditions occur?
 - Between threads
 - Between processes
 - Between main level and interrupt level
 - Between multiple processors
 - Between the CPU and I/O devices
 - Any time two flows of control can access the same storage

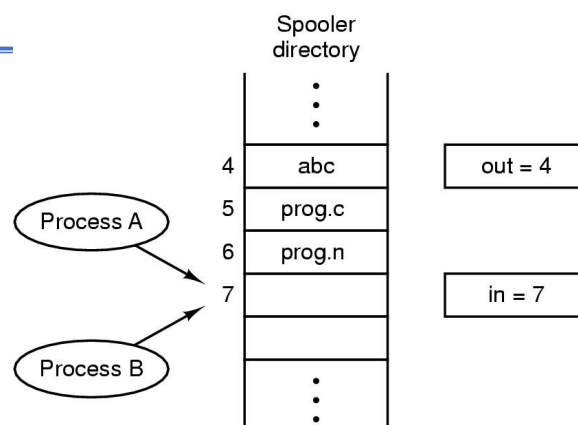
Spring 2026

Operating Systems

7

7

Race Conditions



Two processes want to access shared memory at the same time

Spring 2026

Operating Systems

8

8

Race Conditions are Bad

- System hangs
- System crashes
- Lost data
- Security problems
- Unpredictability

Spring 2026

Operating Systems

9

9

Critical Regions

- The part of a program that uses a shared variable is a critical region
- Two (or more) programs can't be in their critical regions at the same time
- One program (or process or thread or device) has to block if it needs access to the critical region while the other is in its critical region

Spring 2026

Operating Systems

10

10

Conditions Required to Avoid Race Condition

- No two processes may be simultaneously inside their critical regions (safety)
- No assumptions may be made about speeds or the number of CPUs (generality)
- No process running outside its critical region may block other processes (efficiency)
- No process should have to wait forever to enter its critical region (fairness)

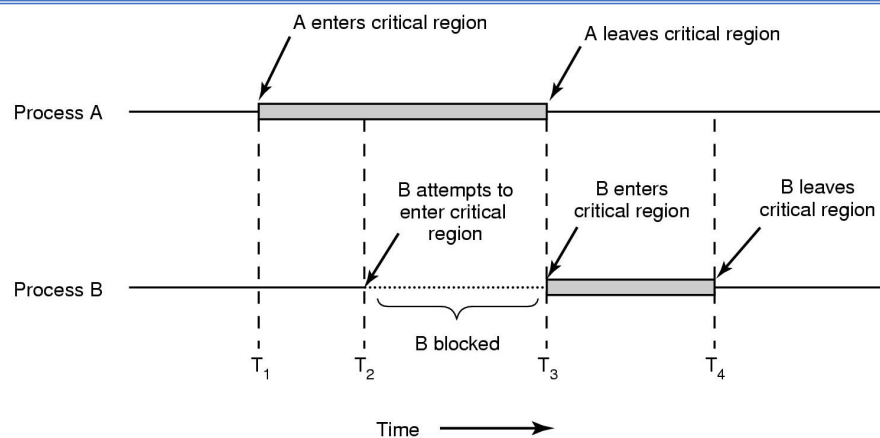
Spring 2026

Operating Systems

11

11

Mutual Exclusion using Critical Regions



Spring 2026

Operating Systems

12

12

Mutual Exclusion with Busy Waiting

- Proposals for achieving mutual exclusion:
 - Disabling interrupts
 - Lock variables
 - Strict alternation
 - Peterson's solution
 - The TSL instruction

Spring 2026

Operating Systems

13

13

Entering a Critical Region

- We have to have mechanisms that mediate access to critical regions
- Control flows must — somehow — signal when they're entering and leaving a critical region
- Access to shared data must not occur outside the critical region

Spring 2026

Operating Systems

14

14

Disabling Interrupts

- Available only to the kernel
- Better not be available to user processes!
- Doesn't work well on multiprocessors
- A special-case solution for the kernel only

15

Lock Variables

- How about this?

```
while (lock != 0)
    ;
/* Start critical region */
lock = 1;
...
/* End critical region */
lock = 0;
```

- Does not work — there is a window between testing for zero and setting it to 1

16

Let's Use a C Feature: ++

```
while(lock++ != 0)
    lock--;
/* critical region ...*/
```

- Is lock++ atomic?
- No — the language makes no guarantees about that!
- IBM mainframe sequence:
 - L R0,lock Fetch variable into Register 0
 - LR R1,R0
 - A R1,=F'1' Increment it
 - ST R1,lock Store new value in 'lock'
 - C R0,=F'0' Compare original value to 0
 - BZ CRITREG If so, go on...
- No atomic "increment" instruction!

17

Test and Set Lock

- We need an atomic test/set instruction
- Some — but not all — architectures have one
- The instruction reads the old value and writes non-zero into the location
- The memory bus is locked during this instruction — even a second CPU can't intervene

18

Using “Test and Set Lock”

```

LOCKTST    TSL R0,lock    Copy lock to R0 and set it non-zero
            CMP R0,#0      Was it zero?
            BNZ LOCKTST    Branch back if non-zero
  
```

Instead of a special instruction, the Pentium has a LOCK prefix that can be applied to several instructions

The TSL Instruction (1)

```

enter_region:
    TSL REGISTER,LOCK      | copy lock to register and set lock to 1
    CMP REGISTER,#0        | was lock zero?
    JNE enter_region       | if it was nonzero, lock was set, so loop
    RET                   | return to caller; critical region entered

leave_region:
    MOVE LOCK,#0           | store a 0 in lock
    RET                   | return to caller
  
```

Entering and leaving a critical region using the TSL instruction

The TSL Instruction (2)

Busy Waiting!

```

enter_region:
    MOVE REGISTER,#1          | put a 1 in the register
    XCHG REGISTER,LOCK        | swap the contents of the register and lock variable
    CMP REGISTER,#0          | was lock zero?
    JNE enter_region          | if it was non zero, lock was set, so loop
    RET                      | return to caller; critical region entered

leave_region:
    MOVE LOCK,#0             | store a 0 in lock
    RET                      | return to caller
  
```

Entering and leaving a critical region using the XCHG instruction

Spring 2026

Operating Systems

21

21

Strict Alternation

Busy Waiting!

```

Process 0                                Process 1                                Spin Lock
while (TRUE) {                            while (TRUE) {
    while (turn != 0)                      while (turn != 1)
        ;
    critical_region();                     critical_region();
    turn = 1;                             turn = 0;
    noncritical_region();                  noncritical_region();
}                                          }
  
```

Spring 2026

Operating Systems

22

22

Disadvantages of Strict Alternation

- Suppose that Process 1's non-critical region code takes a long time to execute
- It won't re-enter its critical region
- Until it does, Process 0 can't re-enter its own critical region
- This violates our efficiency principle

Spring 2026

Operating Systems

23

23

Peterson's Algorithm (1981)

- Simple algorithm using only ordinary (i.e., non-locking) instructions
- Does not require strict alternation – using lock variables and warning variables
- Described in a two-page paper that includes a (simple) proof

Spring 2026

Operating Systems

24

24

Peterson's Algorithm

```

#define FALSE 0
#define TRUE 1
#define N      2          /* number of processes */

int turn;                  /* whose turn is it? */
int interested[N];         /* all values initially 0 (FALSE) */

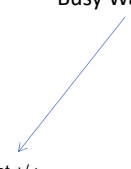
void enter_region(int process); /* process is 0 or 1 */
{
    int other;              /* number of the other process */

    other = 1 - process;    /* the opposite of process */
    interested[process] = TRUE; /* show that you are interested */
    turn = process;         /* set flag */
    while (turn == process && interested[other] == TRUE) /* null statement */ ;
}

void leave_region(int process) /* process: who is leaving */
{
    interested[process] = FALSE; /* indicate departure from critical region */
}

```

Busy Waiting!



Spring 2026

Operating Systems

25

25

Spin Locks

- All three correct solutions — Test and Set Lock, Strict Alternation, Peterson's Algorithm — involve busy waiting
- Also known as a spin lock
- Acceptable for short waits, especially on multiprocessors or when dealing with interrupt contexts
- For general-purpose use, need a solution that permits sleeping

Spring 2026

Operating Systems

26

26

Spin Locks and Priority Inversion

- Suppose we have two processes with different priorities, H and L
- L never runs if H is runnable
- While H is sleeping, the low priority process L grabs the spin lock
- H wakes up and tries to get the lock
- It spins, waiting for L to free it
- But L can't get the CPU, so it doesn't progress
- One example of a deadlock

Spring 2026

Operating Systems

27

27

Blocking instead of Busy Waiting

- Prevent wasting time busy waiting
- Simplest way is sleep+wakeup...
 - System call sleep causes caller to block (suspend until another process wakes it up)
 - Alternatively, sleep+wakeup both have a memory address to match them...
- Let's use this in an example

Spring 2026

Operating Systems

28

28

Classic Problem: Produce-Consumer

- Producer wants to add elements to bounded-size buffer
- If buffer is full, producer must sleep
- Consumer wants to remove items from buffer
- If buffer is empty, consumer must sleep

Spring 2026

Operating Systems

29

29

Producer-consumer Problem w/ Race Condition

```
#define N 100
int count = 0;
```

Producer

```
while (TRUE) {
    item = produce_item();
    if (count == N) sleep();
    insert_item();
    if (count++ == 0)
        wake(consumer);
}
```

Consumer

```
while (TRUE) {
    if (count == 0) sleep();
    item = remove_item();
    if (count-- == N)
        wake(producer);
    consume_item(item);
}
```

Spring 2026

Operating Systems

30

30

Semaphores

- Invented by Dijkstra in 1965
- Two operations: down (sometimes known as P) and up (also known as V)
- **down**: decrements semaphore variable if greater than zero; if 0, process sleeps before doing the decrement
- Note: check, decrement, and sleep are atomic
- **up**: increments semaphore; if a process was sleeping on it, wake it and let it do its decrement
- Generalizes well to multiple users of the semaphore

Spring 2026

Operating Systems

31

31

Using Semaphores

```
semaphore mex = 1;  
down(&mex);  
/* critical region */  
up(&mex);
```

Spring 2026

Operating Systems

32

32

Implementing Semaphores

- Typically done in the kernel
- Mask interrupts while manipulating semaphore
- On multiprocessors, use Test and Set Lock protection as well
- Both of these are for only a few microseconds — not a serious problem

Spring 2026

Operating Systems

33

33

Producer-Consumer with Semaphores

```
#define N 100
int count = 0;
semaphore mutex = 1, empty = N, full = 0;
```

Producer

```
while (TRUE) {
    item = produce_item();
    down(&empty);
    down(&mutex);
    insert_item();
    up(&mutex);
    up(&full);
}
```

Consumer

```
while (TRUE) {
    down(&full);
    down(&mutex);
    item = remove_item();
    up(&mutex);
    up(&empty);
    consume_item(item);
}
```

Spring 2026

Operating Systems

34

34

Mutual Exclusion with Busy Waiting

- Proposals for achieving mutual exclusion:
 - Disabling interrupts
 - Lock variables
 - Strict alternation
 - Peterson's solution
 - The TSL instruction



Spring 2026

Operating Systems

35

35

Two Different Uses of Semaphores

- Counting semaphores — atomic way to manipulate a counter with blocking if 0
 - In the example, empty counts how many slots are free and full is the number of slots that are full
 - Used for sleep/wake when buffer is in the wrong state
- Mutual exclusion semaphores — make sure only one process is in critical region
 - In this case, it protects access to the buffer
 - (Probably, that code should be in the insert item()/ remove item() routines)

Spring 2026

Operating Systems

36

36

Mutexes

- Special case of semaphore: useful when no counting is needed
- If Test and Set Lock instruction is available, easy to implement at user level for thread package
- Try to grab lock; if unsuccessful, let another thread run
- But — if the thread holding the lock is blocked, this thread will burn CPU time until the OS intervenes

Spring 2026

Operating Systems

37

37

Mutexes

mutex_lock:	
TSL REGISTER,MUTEX	copy mutex to register and set mutex to 1
CMP REGISTER,#0	was mutex zero?
JZE ok	if it was zero, mutex was unlocked, so return
CALL thread_yield	mutex is busy; schedule another thread
JMP mutex_lock	try again
ok: RET	return to caller; critical region entered
mutex_unlock:	
MOVE MUTEX,#0	store a 0 in mutex
RET	return to caller

Implementation of mutex lock and mutex unlock (note the subtle difference – adding thread_yield to avoid busy waiting)

Spring 2026

Operating Systems

38

38

Mutexes in Pthreads (1)

Thread call	Description
Pthread_mutex_init	Create a mutex
Pthread_mutex_destroy	Destroy an existing mutex
Pthread_mutex_lock	Acquire a lock or block
Pthread_mutex_trylock	Acquire a lock or fail
Pthread_mutex_unlock	Release a lock

Some of the Pthreads calls relating to mutexes

Spring 2026

Operating Systems

39

39

Mutexes in Pthreads (2)

Thread call	Description
Pthread_cond_init	Create a condition variable
Pthread_cond_destroy	Destroy a condition variable
Pthread_cond_wait	Block waiting for a signal
Pthread_cond_signal	Signal another thread and wake it up
Pthread_cond_broadcast	Signal multiple threads and wake all of them

Some of the Pthreads calls relating to condition variables

Mutexes → good for allowing or blocking access to a critical region
 Condition variables → allow threads to block due to some condition not being met...

Almost always used together... Check and sleep...

Spring 2026

Operating Systems

40

40

Mutexes in Pthreads (3)

```
#include <stdio.h>
#include <pthread.h>
#define MAX 1000000000 /* how many numbers to produce */
pthread_mutex_t the_mutex;
pthread_cond_t condc, condp;
int buffer = 0; /* buffer used between producer and consumer */

void *producer(void *ptr) /* produce data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer != 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = i; /* put item in buffer */
        pthread_cond_signal(&condc); /* wake up consumer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

void *consumer(void *ptr) /* consume data */
{
    int i;
    for (i = 1; i <= MAX; i++) {
        pthread_mutex_lock(&the_mutex); /* get exclusive access to buffer */
        while (buffer == 0) pthread_cond_wait(&condc, &the_mutex);
        buffer = 0; /* take item out of buffer */
        pthread_cond_signal(&condp); /* wake up producer */
        pthread_mutex_unlock(&the_mutex); /* release access to buffer */
    }
    pthread_exit(0);
}

int main(int argc, char **argv)
{
    pthread_t pro, con;
    pthread_mutex_init(&the_mutex, 0);
    pthread_cond_init(&condc, 0);
    pthread_cond_init(&condp, 0);
    pthread_create(&con, 0, consumer, 0);
    pthread_create(&pro, 0, producer, 0);
    pthread_join(pro, 0);
    pthread_join(con, 0);
    pthread_cond_destroy(&condc);
    pthread_cond_destroy(&condp);
    pthread_mutex_destroy(&the_mutex);
}
```

Spring 2026

Operating Systems

41

41

The Risks of Semaphores and Mutexes

- Using semaphores and mutexes correctly is difficult
- If you get it wrong, you can deadlock
- Testing is difficult, because it's all timing-dependent
- We need a higher-level construct

Spring 2026

Operating Systems

42

42

Monitors

- Invented by Hoare (1974) and Brinch Hansen (1975)
- A programming language construct
- Java has it; C and C++ do not
- A monitor is like a class, but only one thread can be executing in it at a time
- In other words, they're a language implementation of mutexes

Spring 2026

Operating Systems

43

43

Monitors (1)

```

monitor example
  integer i;
  condition c;

  procedure producer( );
  .
  .
  end;

  procedure consumer( );
  . . .
  end;
end monitor;

```

Spring 2026

Operating Systems

44

44

Monitors in Java

How is this done?

- Each object may be used as a monitor
- Methods requiring mutual exclusion must be explicitly marked with the **synchronized** keyword
 - Blocks of code may also be marked by synchronized
- No explicit condition variables needed
 - each monitor (i.e. object) is equipped with a single wait queue in addition to its entrance queue.
 - waiting is done on this single wait queue and all notify and notifyAll operations apply to this queue.
- This approach has been adopted in other languages, for example C#

Spring 2026

Operating Systems

45

45

Monitors and Application Blocking

- We still need a way to block if, say, the buffer is full
- Two new operations: wait and signal
- Subtle semantics about who gets to run within the monitor after a signal call
 - Hoare: signaler blocks; waiter runs
 - Brinch Hansen: signal can only be done when exiting the monitor
 - Or: signaler keeps running; waiter resumes when signaler exits monitor

Spring 2026

Operating Systems

46

46

Monitors (2)

```

monitor ProducerConsumer
  condition full, empty;
  integer count;
  procedure insert(item: integer);
  begin
    if count = N then wait(full);
    insert_item(item);
    count := count + 1;
    if count = 1 then signal(empty)
  end;
  function remove: integer;
  begin
    if count = 0 then wait(empty);
    remove = remove_item;
    count := count - 1;
    if count = N - 1 then signal(full)
  end;
  count := 0;
end monitor;

```

```

procedure producer;
begin
  while true do
    begin
      item = produce_item;
      ProducerConsumer.insert(item)
    end
  end;
procedure consumer;
begin
  while true do
    begin
      item = ProducerConsumer.remove;
      consume_item(item)
    end
  end;

```

An outline of the producer-consumer problem with monitors

Spring 2026

Operating Systems

47

47

The Disadvantages of Monitors

- Monitors are nice, but they're only available in a very few languages
- Monitors, semaphores, and mutexes are fine on a single machine, even a multiprocessor
- They don't work well without some shared memory
- What about a distributed system connected via a LAN?

Spring 2026

Operating Systems

48

48

Message Passing

- Model based on network operations:
 - `send(dest, &mesg);`
 - `recv(src, &mesg);`
- Receiver blocks if there's no data available
- Sender blocks if the receiver's buffers are full
- Complex network questions, including acknowledgment, flow control, error detection and correction, message ordering, and authentication
 - all out of scope for this course

Spring 2026

Operating Systems

49

49

Using Message Passing

- Many different paradigms of how to use it
- Rendezvous — one message at a time; sender and receiver operate in lock-step
- Mailboxes — fixed-size buffers on channel; sender blocks if they're full
- Explicit requests — empty messages from the consumer to the producer

Spring 2026

Operating Systems

50

50

Message Passing (1)

```

public class ProducerConsumer {
    static final int N = 100; // constant giving the buffer size
    static producer p = new producer(); // instantiate a new producer thread
    static consumer c = new consumer(); // instantiate a new consumer thread
    static our_monitor mon = new our_monitor(); // instantiate a new monitor

    public static void main(String args[]) {
        p.start(); // start the producer thread
        c.start(); // start the consumer thread
    }

    static class producer extends Thread {
        public void run() { // run method contains the thread code
            int item;
            while (true) { // producer loop
                item = produce_item();
                mon.insert(item);
            }
        }
        private int produce_item() { ... } // actually produce
    }
}

```

...

A solution to the producer-consumer problem in Java

Spring 2026

Operating Systems

51

51

Message Passing (2)

...

```

static class consumer extends Thread {
    public void run() { // run method contains the thread code
        int item;
        while (true) { // consumer loop
            item = mon.remove();
            consume_item(item);
        }
    }
    private void consume_item(int item) { ... } // actually consume
}

static class our_monitor { // this is a monitor
    private int buffer[] = new int[N];
    private int count = 0, lo = 0, hi = 0; // counters and indices

    public synchronized void insert(int val) {
        if (count == N) go_to_sleep(); // if the buffer is full, go to sleep
        buffer[hi] = val; // insert an item into the buffer
        hi = (hi + 1) % N; // slot to place next item in
        count = count + 1; // one more item in the buffer now
        if (count == 1) notify(); // if consumer was sleeping, wake it up
    }
}

```

...

A solution to the producer-consumer problem in Java

Spring 2026

Operating Systems

52

52

Message Passing (3)

...

```

public synchronized int remove() {
    int val;
    if (count == 0) go_to_sleep(); // if the buffer is empty, go to sleep
    val = buffer[lo]; // fetch an item from the buffer
    lo = (lo + 1) % N; // slot to fetch next item from
    count = count - 1; // one fewer items in the buffer
    if (count == N - 1) notify(); // if producer was sleeping, wake it up
    return val;
}
private void go_to_sleep() { try{wait();} catch(InterruptedException exc) {};
}
}

```

A solution to the producer-consumer problem in Java

Spring 2026

Operating Systems

53

53

Producer-Consumer Problem w/ MP (1)

```

#define N 100 // number of slots in the buffer */

void producer(void)
{
    int item;
    message m; // message buffer */

    while (TRUE) {
        item = produce_item(); // generate something to put in buffer */
        receive(consumer, &m); // wait for an empty to arrive */
        build_message(&m, item); // construct a message to send */
        send(consumer, &m); // send item to consumer */
    }
}

```

...

The producer-consumer problem with N messages.

Spring 2026

Operating Systems

54

54

Producer-Consumer Problem w/ MP (2)

```

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */
    while (TRUE) {
        receive(producer, &m);                /* get message containing item */
        item = extract_item(&m);                /* extract item from message */
        send(producer, &m);                    /* send back empty reply */
        consume_item(item);                    /* do something with the item */
    }
}

```

The producer-consumer problem with N messages.

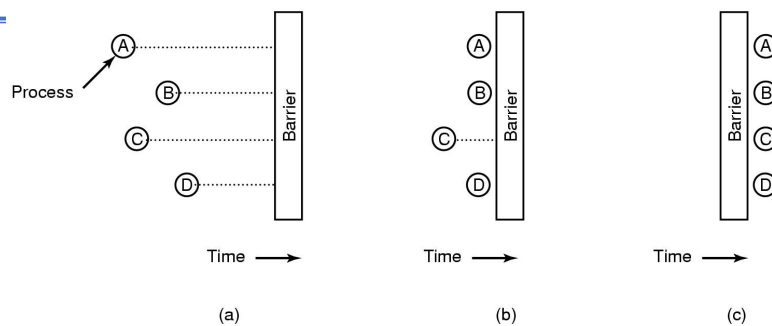
Spring 2026

Operating Systems

55

55

Barriers (for Multi-Processes Synchron.)



Processes with multiple phases: check at the end of each phase...

(a) Processes approaching a barrier

(b) All processes but one blocked at the barrier

(c) When the last process arrives at the barrier, all of them are let through

Spring 2026

Operating Systems

56

56

Summary

- IPC can be hard
- Many possible schemes, depending on environment and underlying OS
- Danger of deadlock



Spring 2026

Operating Systems

57

57

See you next time!

58